

Chapitre 2 Partie 3 :

Les Fonctions en JavaScript

Table des matières

<u>1.</u>	<u>Vue d'ensemble :</u>	3
a.	<u>Syntaxe</u>	3
b.	<u>Exemple</u>	3
c.	<u>Omission des ()</u>	4
d.	<u>Créer des fonctions</u>	4
e.	<u>Utilisation d'une déclaration de fonction</u>	4
f.	<u>Utilisation d'une expression de fonction</u>	5
g.	<u>Comparaison de la déclaration de fonction et de l'expression de fonction</u>	6
<u>2.</u>	<u>Paramètres de fonction et arguments</u>	6
h.	<u>Différence entre paramètre et argument</u>	6
i.	<u>Exemple</u>	7
<u>3.</u>	<u>Paramètres manquants et valeur non définie</u>	7
<u>4.</u>	<u>Tableau d'arguments</u>	7
<u>5.</u>	<u>Simulation du passage de paramètre optionnel (arguments facultatifs)</u>	8
<u>6.</u>	<u>L'instruction return</u>	8
<u>7.</u>	<u>Fonction appelant une autre</u>	9
<u>8.</u>	<u>Passage de paramètres par valeur ou par référence</u>	9
j.	<u>Exemple de passage par valeur</u>	9
k.	<u>Exemple de passage par référence</u>	10
<u>9.</u>	<u>Nommage des fonctions</u>	10
<u>10.</u>	<u>Les fonctions fléchées</u>	10
•	<u>Cas d'utilisation des fonctions fléchées</u>	11
<u>11.</u>	<u>Quelques fonctions prédéfinies</u>	12
l.	<u>eval</u>	12
m.	<u>isFinite</u>	13
n.	<u>isNaN</u>	13
o.	<u>parseFloat</u>	13
p.	<u>parseInt</u>	13

q.	Number	13
r.	String	14
s.	Escape	14

1. Vue d'ensemble :

Une fonction peut être simplement définie comme un ensemble d'instructions ou un sous-programme qui peut être utilisé ou appelé n'importe où dans le programme JS.

Une fonction a généralement un nom, mais n'a pas besoin d'avoir de nom, auquel cas elle s'appelle une fonction anonyme. Une fonction peut avoir zéro ou plusieurs paramètres.

Une fonction a un corps entre des accolades, et le corps de la fonction peut inclure plusieurs instructions.

Une fonction peut également inclure une instruction `return`, qui est facultative et peut être utilisée pour renvoyer au code appelant une valeur calculée.

Le traitement de la fonction s'arrête à l'instruction `return`.

L'utilisation de la fonction offre les avantages suivants aux développeurs :

- Elle économise beaucoup de temps et d'efforts car le développeur n'a pas besoin de réécrire le code pour une exigence particulière.
- Le code du programme est plus compact, plus organisé et plus compréhensible qu'il ne le serait sans les fonctions.

Une fois les fonctions définies une seule fois, elles peuvent être utilisées dans tout le programme ou dans d'autres fonctions.

Les fonctions peuvent être créées principalement de différentes manières sous forme **d'expression de fonction** ou en utilisant une **déclaration de fonction** ou encore en utilisant **le mot clé Function** permettant de créer un objet fonction .

a. Syntaxe

```
function nomfonction (listeparamètresseprespardevirgules)
{
    //corps de la fonction
    // return est facultative et peut être utilisée pour renvoyer au code appelant
    une valeur
}
```

Si la fonction **n'a pas de paramètres**, vous devez toujours inclure les parenthèses après le nom de la fonction :

```
function nomfonction()
{
    //Instructions
}
```

a. Exemple

```
function triple (num)
{
    return (3 * num);
}
var x = triple(4);
```

La fonction est appelée, la valeur de retour se retrouve dans x

```
var x = triple(4);
```

Une fonction peut être utilisée dans une fonction d'alerte comme suit :

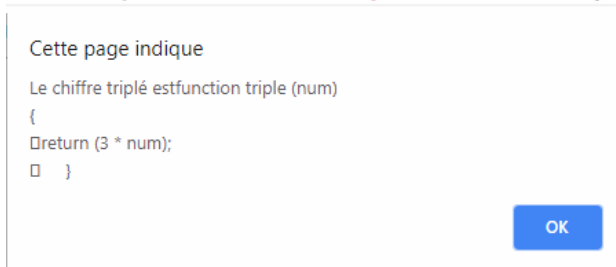
```
alert ('Le chiffre triplé est' + triple (2));
```

b. Omission des ()

Omettre l'opérateur () conduit à renvoyer le code de la fonction.

Exemple

```
alert ('Le chiffre triplé est' + triple);
```



c. Créer des fonctions

Les fonctions peuvent être créées de trois manières :

- Utilisation d'une déclaration de fonction
- Utilisation d'une expression de fonction
- Une autre manière de créer une fonction, qu'on détaillera dans le chapitre sur les objets, est l'utilisation de l'objet Function.

Note : Il est préférable d'utiliser une expression de fonction ou une déclaration de fonction que l'objet Function.

d. Utilisation d'une déclaration de fonction

Une déclaration de fonction définit une fonction avec un nom et n'implique aucune affectation de variable.

Les fonctions déclarées ne sont exécutées que lorsqu'elles sont appelées et ne sont pas exécutées immédiatement.

Syntaxe

```
function nomf (paramètres)
{instructions}
```

Exemple

```
function myFunction ()
{
return 'Bonjour';
}
alert ("Texte affiché en tant que :" + myFunction ());
```

Notez qu'une déclaration de fonction n'est pas une instruction exécutable. Il n'est pas nécessaire d'avoir un point-virgule à la fin d'une déclaration de fonction.

e. Utilisation d'une expression de fonction

Une fonction définie via une expression de fonction peut être de trois types :

- Fonctions auto-appelantes
- Expression de fonction nommée
- Expression de fonction anonyme

i. Fonctions auto-appelantes (auto exécutables)

Une fonction auto-appelante est une fonction qui peut être définie avec un nom ou sans nom mais qui est invoquée automatiquement. Il n'y a pas de déclaration et d'invocation impliquées. L'expression exécute la fonction automatiquement.

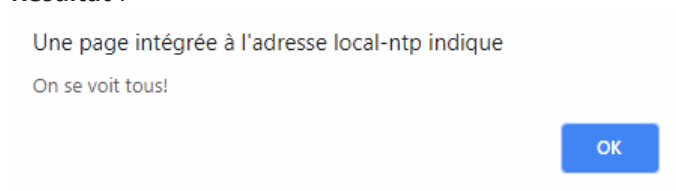
Syntaxe

```
(function () { // code }) ();
```

Exemple

```
(function () { alert ("On se voit tous!"); }) ();
```

Résultat :



ii. Expression de fonction nommée

Comme son nom l'indique, la fonction nommée est déclarée avec un nom dans une expression de fonction.

Syntaxe

```
var myF = function nomF () {instructions}
```

Ici, une variable myF est définie, à laquelle l'objet fonction est affecté. Vous pouvez ensuite utiliser la variable myF et fournir les valeurs relatives à la fonction, le cas échéant, afin d'exécuter le code de la fonction nomF.

Exemple

```
var myValue = function myFunction () {return 'Bonjour'; }
alert ("Texte affiché en tant que:" + myValue ());
```

iii. Fonction anonyme (sans nom)

Une fonction (expression) qui n'a pas de nom est appelée une fonction anonyme.

Syntaxe

```
var myF = function () {instructions}
```

Exemple

```
var salut = function () {return 'Bonjour'; }
alert ("Texte affiché en tant que:" + salut ());
```

f. Comparaison de la déclaration de fonction et de l'expression de fonction

Ceci est une déclaration :	Ceci est une expression :
<code>function produit (a, b) {return a * b; }</code>	<code>var produit = function (a, b) {return a * b; }</code>

1. Paramètres de fonction et arguments

g. Différence entre paramètre et argument

Il peut y avoir un ou plusieurs paramètres pour une fonction donnée. Sur la base des valeurs de paramètres fournies à la fonction, le comportement de la fonction change et les résultats sont calculés en conséquence.

Les paramètres tels que a et b peuvent être adressés comme n'importe quelle variable locale dans le code de la fonction.

```
function unexemple (p1, p2)
{...}
```

Au moment de l'appel de la fonction, les valeurs correspondantes fournies pour les paramètres sont appelées arguments. Par exemple :

```
unexemple (10, 11); //10 et 11 sont des arguments
```

h. Exemple

```
<html> <body>
<script>
  function mymessage (utilisateur)
  {
    alert ("Heureux de vous voir" + utilisateur);
  }
  mymessage ("Flen Fouléni") ;
</script>
</body> </html>
```

2. Paramètres manquants et valeur non définie

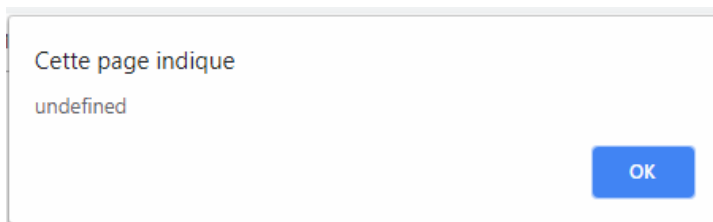
Il est possible d'appeler une fonction multi- paramètres et d'omettre quelques paramètres.

Dans la fonction, tous les paramètres manquants ont la valeur définie undefined. Tous les arguments supplémentaires fournis sont ignorés.

Exemple

```
function myFunction (firstparam, secondparam)
{
    if (secondparam === undefined)
    {
        alert (secondparam)
    }
}
myFunction (1);
```

Résultat :



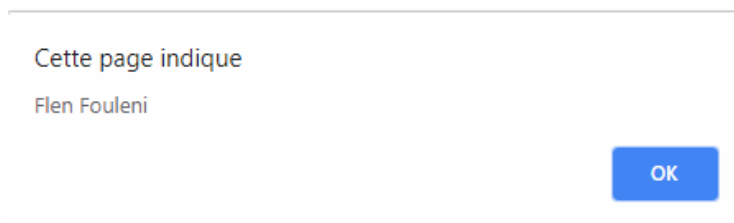
3. Tableau d'arguments

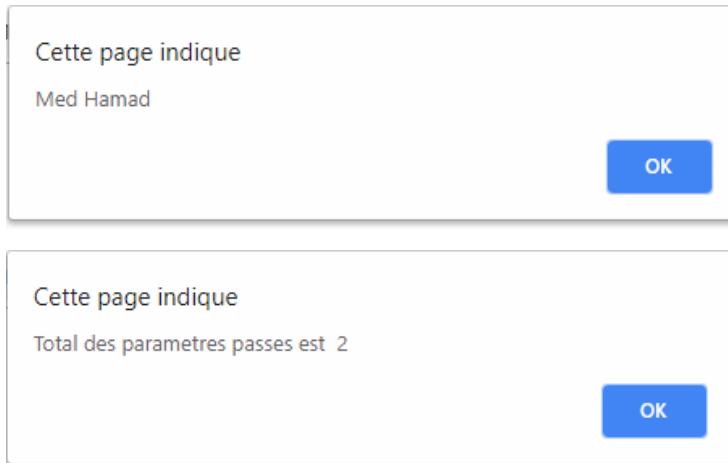
Une fonction peut inclure un tableau spécial nommé arguments qui contient tous les arguments fournis via l'appel de fonction. Ce tableau contient une valeur même si aucun paramètre n'est défini dans la définition de la fonction.

Exemple

```
function totparam ()
{
    alert(arguments [0]);
    alert(arguments [1]);
    alert('Total des paramètres passés sont' + arguments.length)
}
totparam('Flen Fouleni', 'Med Hamad');
```

Résultat :





4. Simulation du passage de paramètre optionnel (arguments facultatifs)

Il est également possible de rendre les paramètres optionnels dans les fonctions.

Exemple

```
function myDivision (nombre, diviseur)
{
  if (diviseur === undefined)
  {
    return ('Division par zéro impossible');
  }
  return nombre / diviseur;
}
alert (myDivision (8)); //retourne Le message 'Division par zéro impossible'
alert (myDivision (100, 10)); //retourne 10
```

5. L'instruction return

L'instruction return est requise si vous souhaitez renvoyer une valeur d'une fonction.

Exemple :

```
<html>
<body>
  <script>
    var resultat = function (nombre, diviseur)
    { return nombre / diviseur; }
    alert (resultat (150, 10)) ;
  </script>
</body>
</html>
```

6. Fonction appelant une autre

```

<html> <body>
<script> function total (a, b)
{
    return a + b;
}
function moyenne (a, b) {
    var av = total(a, b) / 2;
    alert("Moy= " + av);
}
moyenne(8, 8);
</script> </body></html>

```

7. Passage de paramètres par valeur ou par référence

En JavaScript, vous ne pouvez pas passer de paramètres par référence. Lorsque vous passez une variable à une fonction, JS effectue une copie du paramètre et travaille dessus.

Par conséquent, dans la fonction, la valeur du paramètre transmis ne peut pas être modifiée.

Les arguments sont passés par valeur. Si la fonction modifie la valeur d'un argument, ce changement ne se répercute pas en dehors de la fonction.

Cas spécifique : passage par référence d'un objet. Si la fonction modifie les propriétés de l'objet de la référence, ce(s) changement(s) seront perceptibles en dehors de la fonction.

i. Exemple de passage par valeur

```

function decNumber (countVal)
{countVal--; }
var i = 10;
decNumber (i);
alert ("La valeur après la tentative est:" + i);

```

Résultat :



Une façon indirecte pour JS d'effectuer une modification d'une valeur passée à la fonction est via l'utilisation de tableaux.

j. Exemple de passage par référence

```
function decNumber (countVal)
{countVal [0] --; }
var i = [10];
decNumber (i);
alert ("Valeur décréementée à:" + i [0]);
```

8. Nommage des fonctions

Ne nommez jamais votre fonction par le terme function. Ceci est strictement interdit. Considérez le code suivant :

```
> function function (countVal)
    {countVal-- ; }
```

✖ Uncaught SyntaxError: Unexpected token function

VM25:1

```
> |
```

9. Les fonctions fléchées

Les fonctions fléchées sont une syntaxe plus simple et concise pour créer des fonctions . Elles ont été introduites avec ES6 (ES2015), c'est aujourd'hui la fonctionnalité de ES6 la plus utilisée.

Pour définir une fonction on utilise une flèche =>

a. Syntaxe

Nomfunc= (paramètres) => { instructions }

b. Exemples

- Une fonction fléchée avec plusieurs paramètres:

```
// ES6
var sommeES6 = (x, y) => { return x + y }
alert(sommeS6(5, 6)) // 11
```

```
// ES5
var somme = function(x, y) {
    return x + y
}
alert(somme(5, 4)) // 9
```

Lorsque la fonction renvoie une seule instruction les accolades et le mot clé return ne sont pas obligatoires,

Les fonctions fléchées permettent un implicite return (retour implicite), c'est à dire pas besoin du mot clé return , on peut donc écrire :

- **Une fonction avec un seul paramètre:**

<pre>// ES6 const hsalut = name => `Hello \${name}` alert(hsalut('Foulen')) // Hello Foulen</pre>	<pre>// ES5 const salutation = function(name) { return `Hello \${name}` } alert(salutation('Monsieur')) // Hello Monsieur</pre>
--	---

Quand on a un seul paramètre, nous n'avons donc pas besoin des parenthèses.

- **Une fonction sans paramètre:**

```
const fh = () => 'Hello World!'
console.log(fh()) // Hello World!
```

Pour définir une fonction sans paramètre, on met juste des parenthèses vides.

- **Fonctions qui renvoient un littéral d'objets:**

```
const setContent = function(t, c) {
  return {
    title: t,
    content: c
  }
}
console.log(setContent('Hello', 'World!'))
```

```
const setfContent = (t, c) => ({title: t, content: c})
console.log(setfContent('Hello', 'ES6'))
```

Pour retourner un objet, il y a une petite différence, parce que l'objet doit être dans des accolades, il faut donc mettre le return entre parenthèses cette fois-ci pour faire la différence, et nous avons le même type de retour, un objet.

- **Cas d'utilisation des fonctions fléchées**

L'une des utilisations des fonctions fléchées est la manipulation de tableau.

c. Exemples

// Une fonction fléchée vide renvoie **undefined**

```
let vide = () => {};
```

// Une fonction fléchée simple renvoie le minimum entre a et 15

```
var simple = a => a > 15 ? 15 : a;
```

```

    alert(simple(16)); // 15
    alert(simple(10)); // 10
// Une fonction fléchée max
    var Max = (a, b) => {
        if (a > b) {
            return a;
        } else {
            return b;
        }
    }
    alert(Max(10,3)) ;//10
// Soit un tableau de prix . Filtrer ce tableau et retourner les éléments supérieurs à 10.
var prix = [4, 5, 14, 20, 45, 3, 8, 15]//présentation JSON d'un tableau .On peut utiliser new
    var sup10 = prix.filter(p => p >= 10)
    alert(sup10) // [14, 20, 45, 15]
// Soit un tableau d'entiers . Filtrer ce tableau et retourner les entiers pairs.
var arr = [5, 6, 13, 0, 1, 18, 23];
    var even = arr.filter(v => v % 2 == 0);
    alert(even) ;// [6, 0, 18]
//double est un tableau contenant les doubles des éléments de arr
    var double = arr.map(v => v * 2);
    // [10, 12, 26, 0, 2, 36, 46]

```

10. Quelques fonctions prédéfinies

k. eval

Cette fonction exécute un code Javascript à partir d'une chaîne de caractères.

```

<html> <body>
<script>
function evaluation() {
document.formulaire.calcul.value=eval(document.formulaire.saisie.value); }
</script>
<FORM NAME="formulaire">
Saisissez une expression mathématique :
<INPUT TYPE="text" NAME=saisie MAXLENGTH=40 SIZE=40>
<INPUT TYPE="button" VALUE="evaluation." onClick="evaluation()">
<INPUT TYPE="text" NAME=calcul MAXLENGTH=40 SIZE=40>
</FORM>...
</body></html>

```

Résultat

Saisissez une expression mathématique :

l. isFinite

Cette fonction Détermine si le paramètre est un nombre fini.

Renvoie false si ce n'est pas un nombre ou l'infini positif ou infini négatif.

m. isNaN

Cette fonction détermine si le paramètre n'est pas un nombre

(NaN : Not a Number).

Exemple (isFinite isNaN)

```
isFinite(240) //retourne true
isFinite("Un nombre") //retourne false
isNaN("un nombre") //retourne true
isNaN(20) //retourne false
```

n. parseFloat

Cette fonction analyse une chaîne de caractères et retourne un nombre décimal.

Si l'argument évalué n'est pas un nombre, renvoie NaN (Not a Number).

o. parseInt

Cette fonction analyse une chaîne de caractères et retourne un nombre entier de la base spécifiée.

La base peut prendre les valeurs 16 (hexadécimal), 10 (décimal), 8 (octal), 2 (binaire).

Exemple parseInt et parseFloat

```
var numero="125";
var nombre=parseFloat(numero); //retourne le nombre 125
var prix=30.75;
var arrondi = parseInt(prix, 10); //retourne 30
```

p. Number

Cette fonction convertit l'objet spécifié en valeur numérique

q. String

Cette fonction convertit l'objet spécifié en chaîne de caractères

r. Escape

Cette fonction retourne la valeur hexadécimale à partir d'une chaîne de caractère codée en ISO-Latin-1.

Exemple Number String Escape

```
var jour = new Date("December 17, 1995 03:24:00");//converit la date en millisecondes
alert (Number(jour));
jour = new Date(430054663215);//Convertit Le nombre en date Mois jour, Année etc.
alert (String(jour));
escape("!&") //retourne %21%26%
var jour = new Date("December 17, 1995 03:24:00");//converit la date en millisecondes
alert (Number(jour));
jour = new Date(430054663215);//Convertit Le nombre en date Mois jour, Année etc.
alert (String(jour));
escape("!&") //retourne %21%26%
```